

Многопоточность

Содержание урока

- ★ Многопоточность
- ★ Поток. Библиотека `threading`
- ★ Многопоточность. Скорость работы
- ★ Класс `Timer`
- ★ Синхронизация потоков



ВАСИЛИСА СЕРЯКОВА

Data Scientist @Blockchain.com

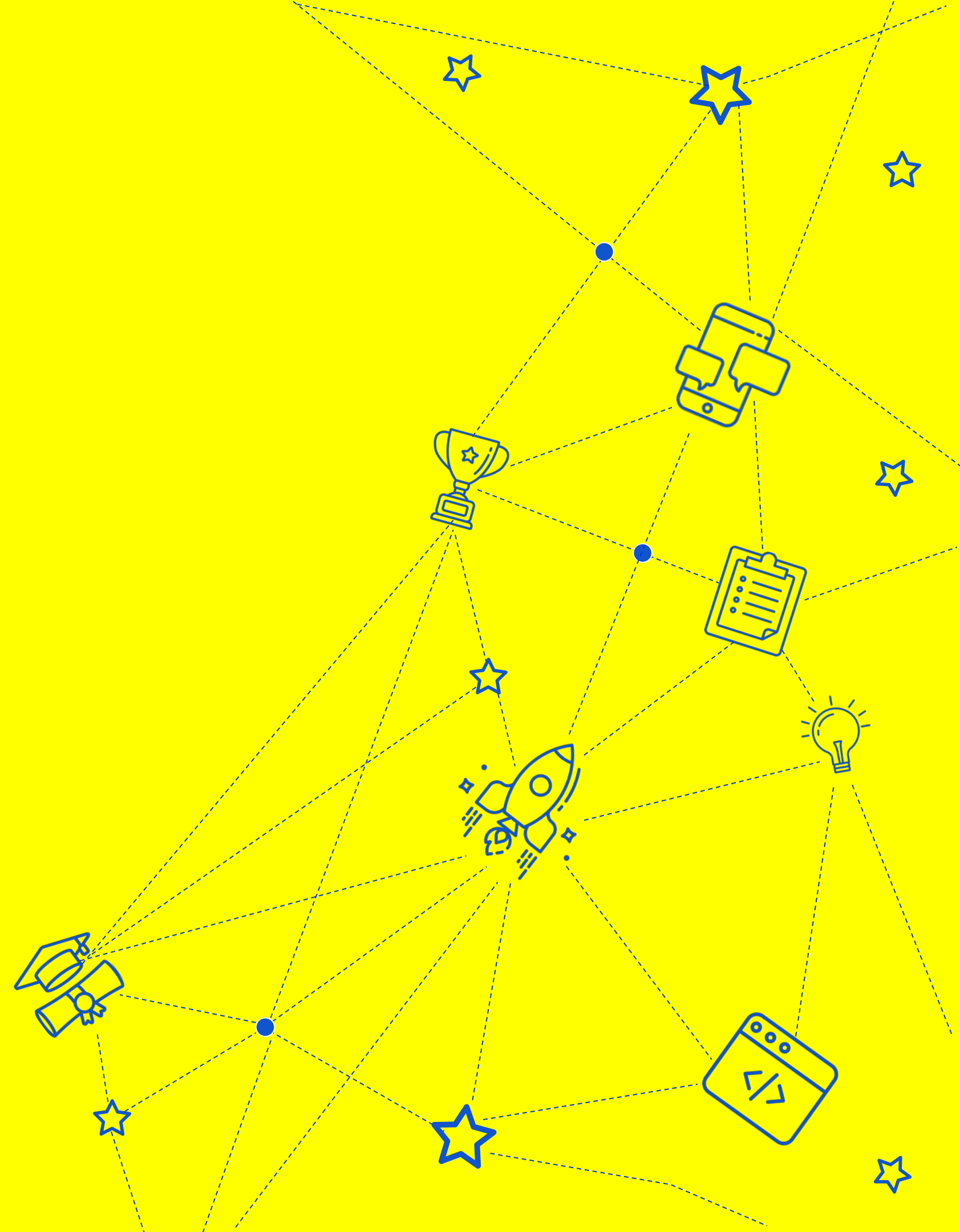
ex-Data Scientist @Yandex



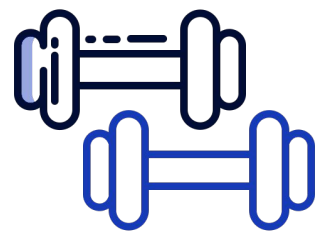
Яндекс  Маркет

- Занимаюсь **Data Engineering** в **Blockchain.com**, строю пайплайны для автоматической проверки транзакций
- Почти 3 года делала **Матчинг** в **Яндекс.Маркете**
- 1 год преподавала в **НИУ ВШЭ** курсы по **Python** и **Машинному обучению**

Многопоточность

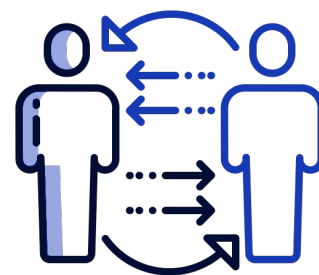


Конкурентность и параллелизм



Конкурентность предполагает выполнение нескольких задач одним исполнителем.

Например, один человек готовит сложное блюдо.

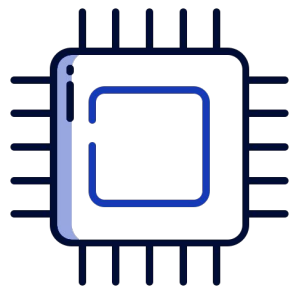


Параллелизм предполагает параллельное выполнение задач разными исполнителями.

Например, вычисление значения функции:

$f(4)$, где $f(x) = x^2 + 2 \cdot x$

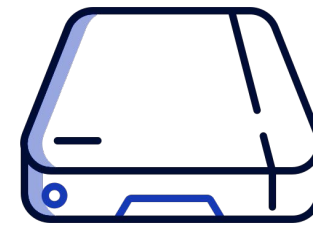
Два типа процессинга: CPU-bound и I/O-bound



CPU-bound

CPU – central processing unit

операции завязанные
исключительно на мощности
процессора



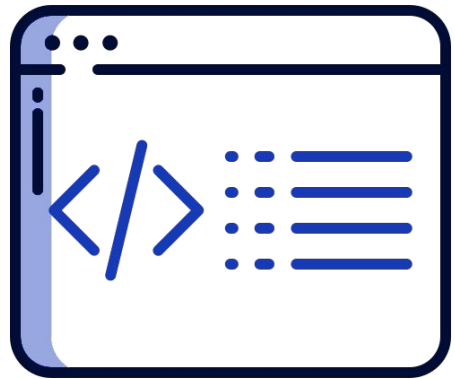
I/O-bound

I/O – Input/Output – Ввод/Вывод

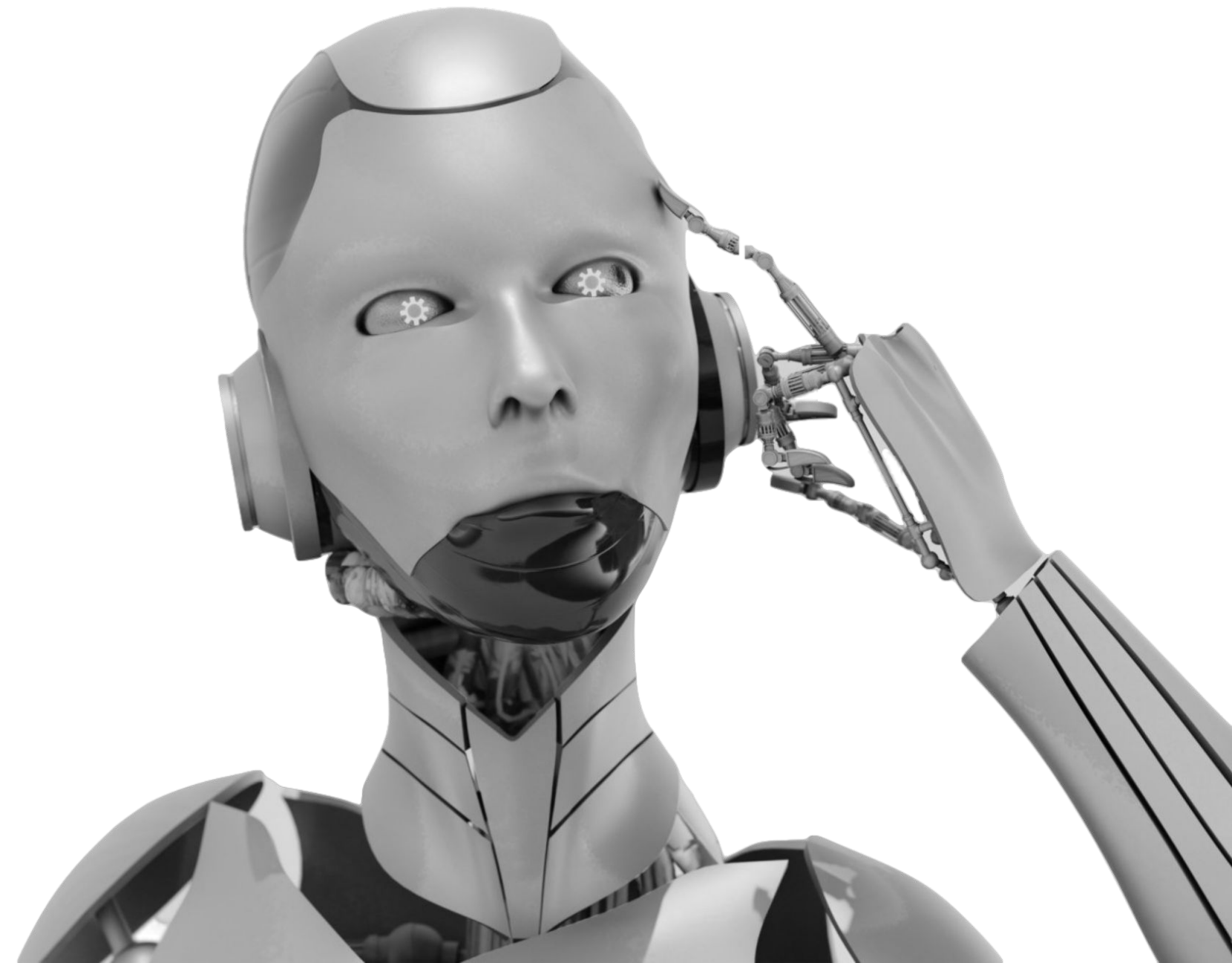
операции завязанные на более
медленных устройствах компьютера
чем CPU, например SSD диск

Несколько слов о GIL

GIL – это аббревиатура от **Global Interpreter Lock** – глобальная блокировка интерпретатора.



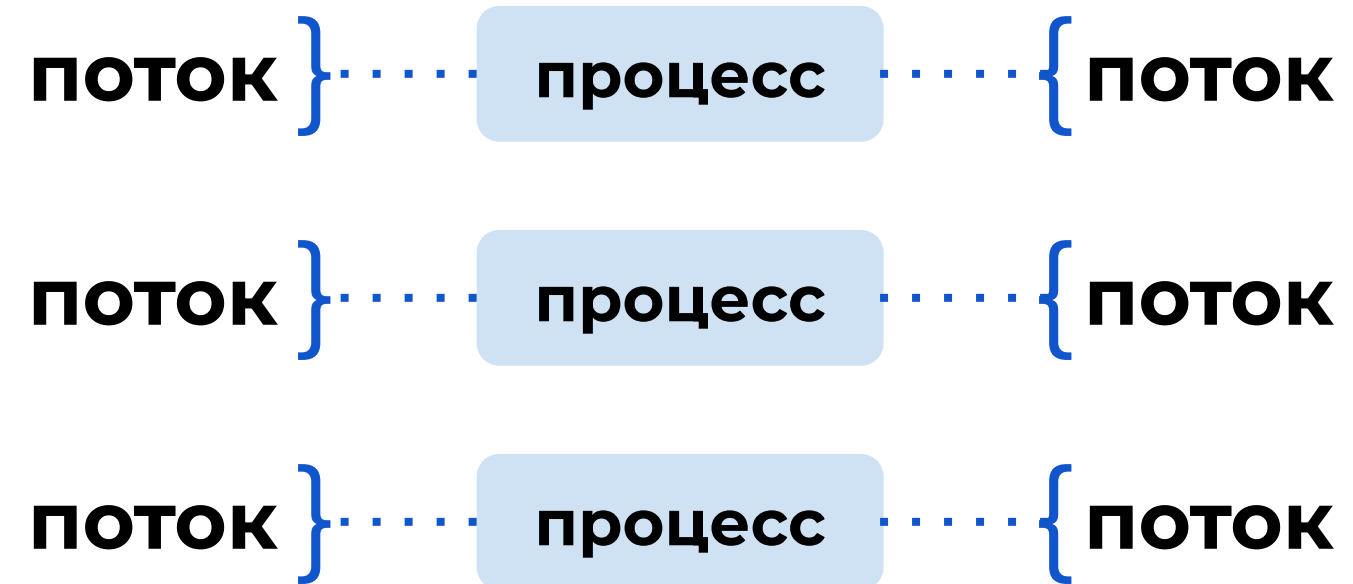
Суть **GIL** в том, что за единицу времени может выполняться только один код.



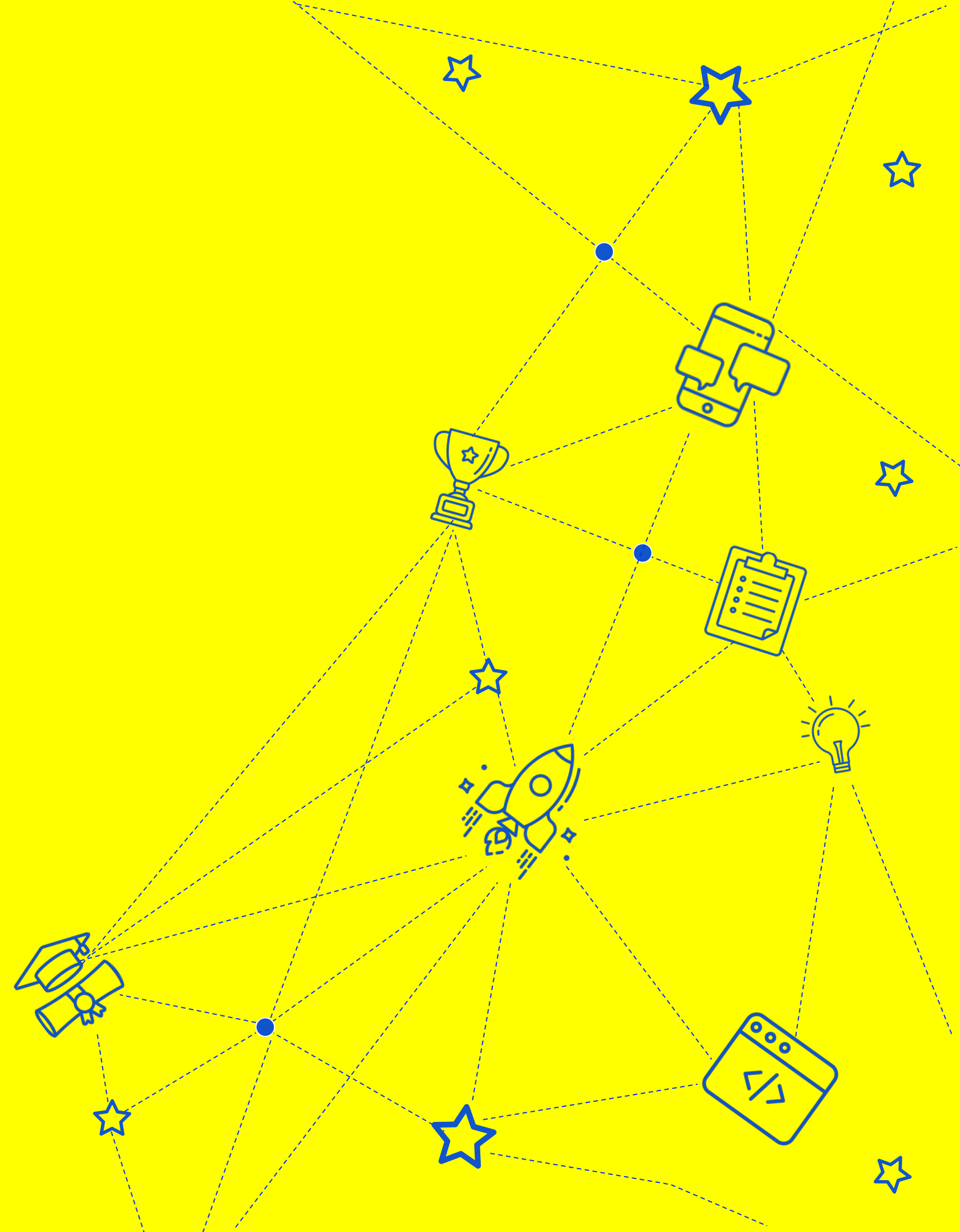
Однопоточная среда



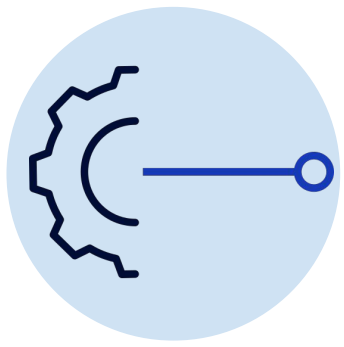
Многопоточная среда



Потоки. Библиотека threading



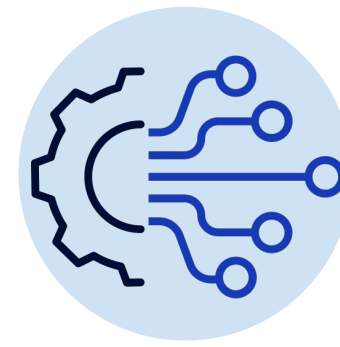
Какие типы потоков существуют



Foreground поток –
главный поток.

Этот тип потоки получают по умолчанию, при их создании.

Особенность этого типа потоков в том, что программа **не завершится**, пока один из **Foreground** потоков будет активен.



Background поток –
второстепенный поток.

Его особенность в следующем: если к моменту завершения последнего **Foreground** потока остаются незавершенные **Background** потоки, программа **все равно завершится**.

Пример кода с базовой реализацией функции

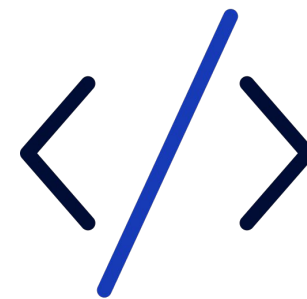
```
import time

def func(x):
    start_time = time.time()
    lst = []
    for elem in range(x):
        lst.append(elem)

    end = time.time()
    print(f'Выполнено за {end - start}')
    return sum(lst)

if __name__ == '__main__':
    #Заход в главный поток по умолчанию
    print('Запуск потока main()')
    print(func(30000000))
    print('Завершение потока main()')
```

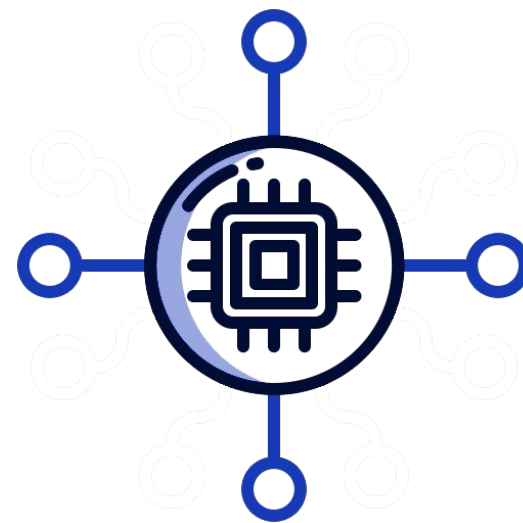
```
Запуск потока main()
Выполнено за 1.6285009384155273
449999985000000
Завершение потока main()
```



- ★ классический вариант
- ★ по умолчанию создается поток **main()**
- ★ это единственный и главный поток **main()** – **Foreground** поток
- ★ программа не завершится до тех пор, пока не будут выполнены все функции внутри потока **main()**

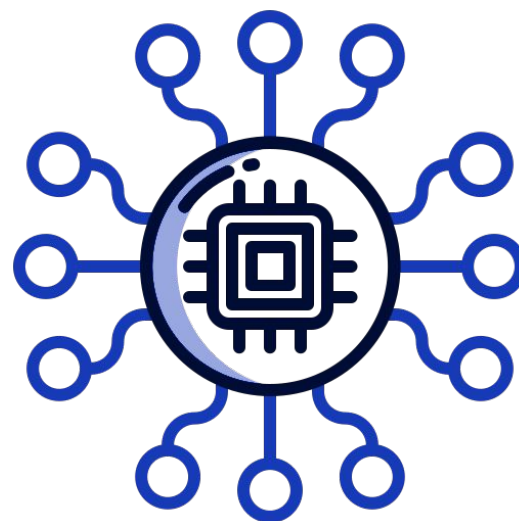
Рассмотрим в виде кода

Как создавать потоки типа Foreground

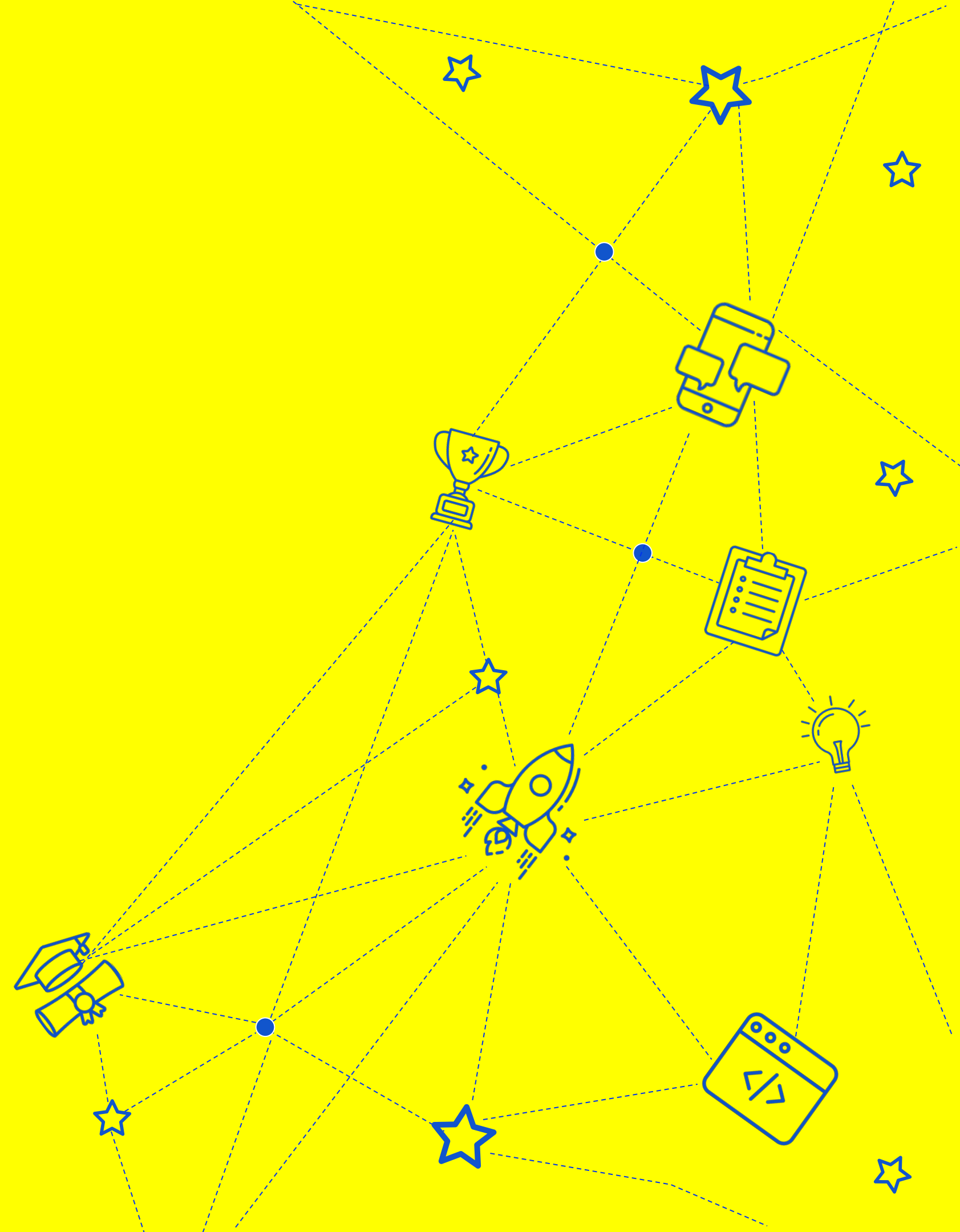


Реализуем в виде кода

Как создавать потоки типа Background



**Многопоточность.
Скорость работы**

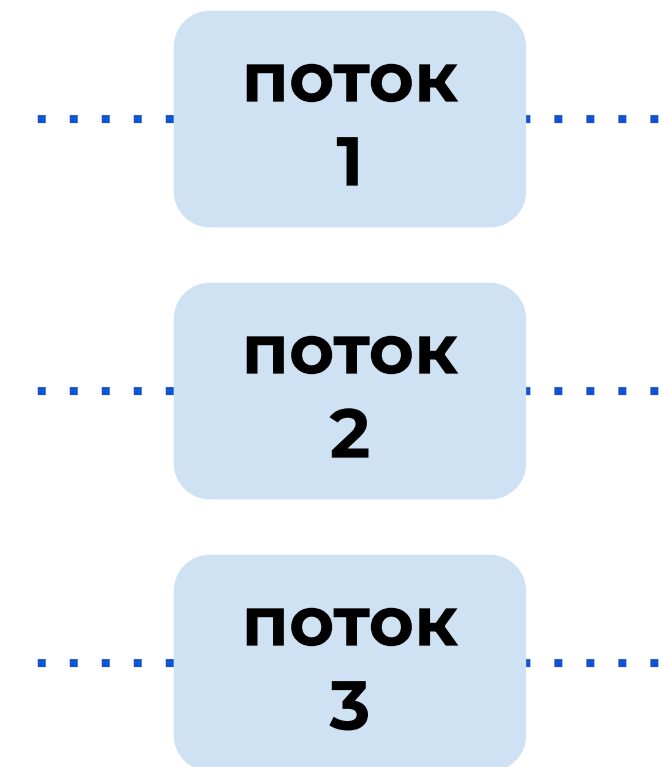


Параллельные и последовательные ПОТОКИ

Последовательные потоки



Параллельные потоки



**Параллельные потоки
работают медленнее
последовательных.**



**Все дело
context switch**

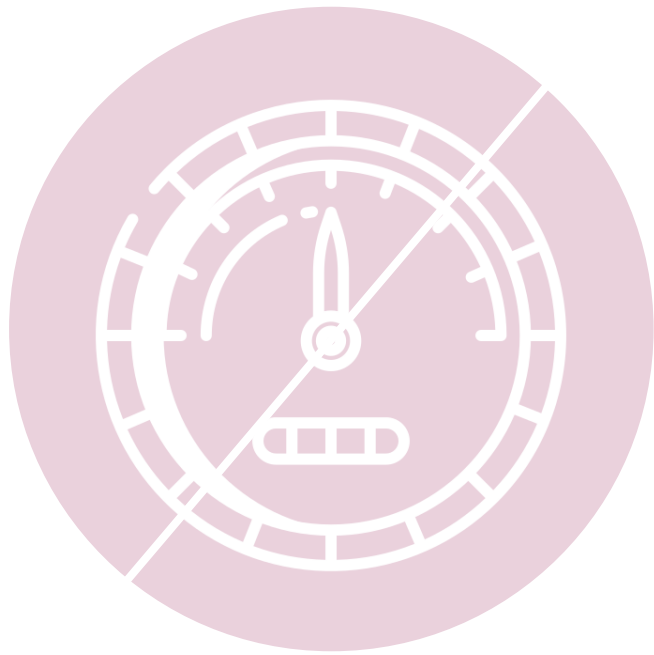
В



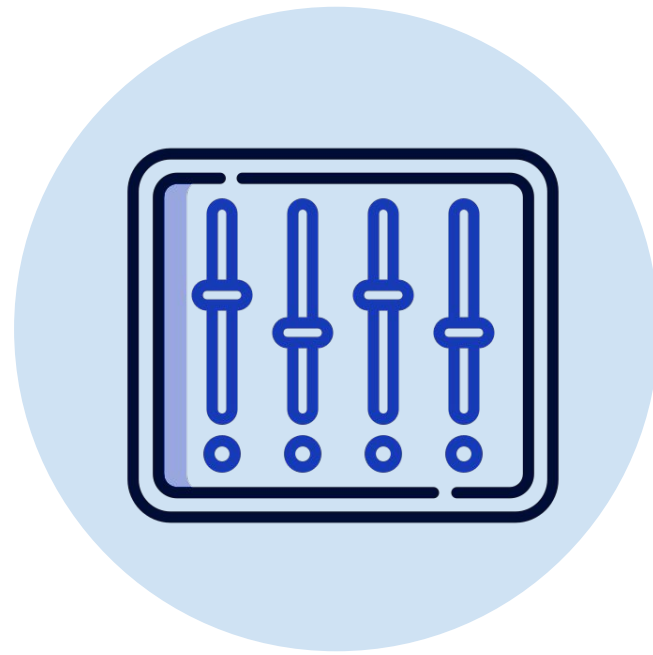
Посмотрим на пример кода



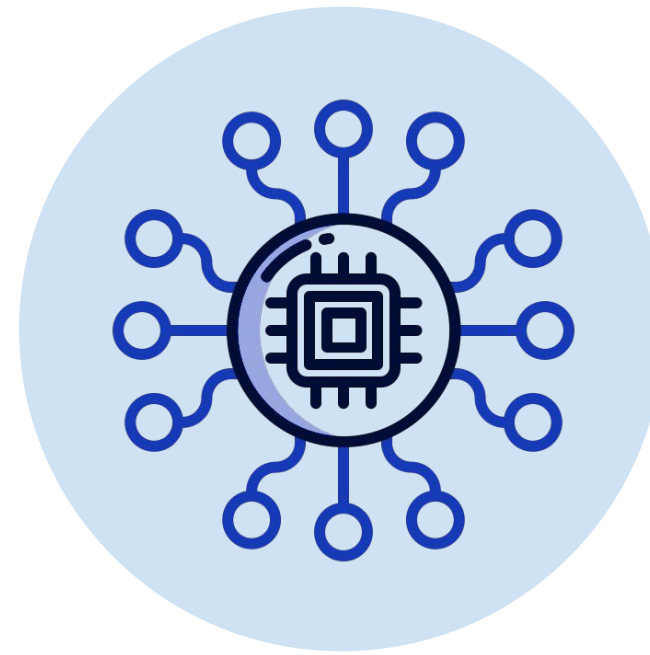
Цели использования параллельных потоков



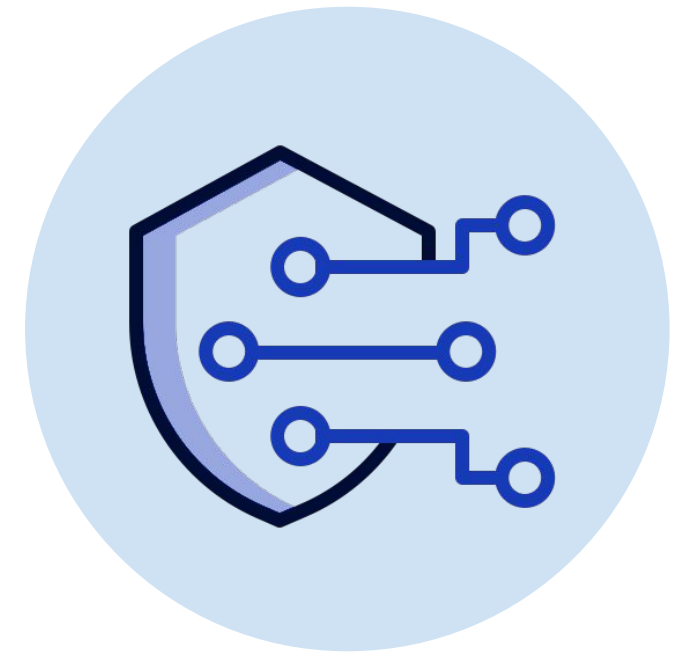
ускорение
работы
программы



всевозможная
синхронизация
этих потоков



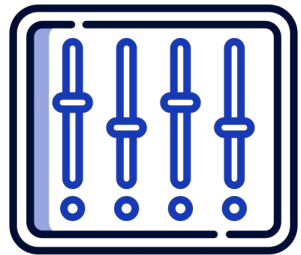
запуск разных
задач внутри
разных потоков



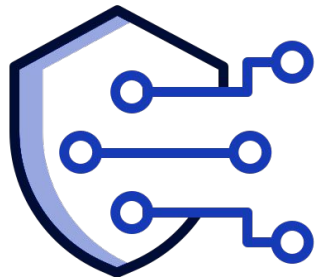
контроль
над порядком
выполнения
потоков

Цели использования параллельных потоков

В следующих модулях мы с вами разберем несколько классов для работы с потоками, которые позволяют:



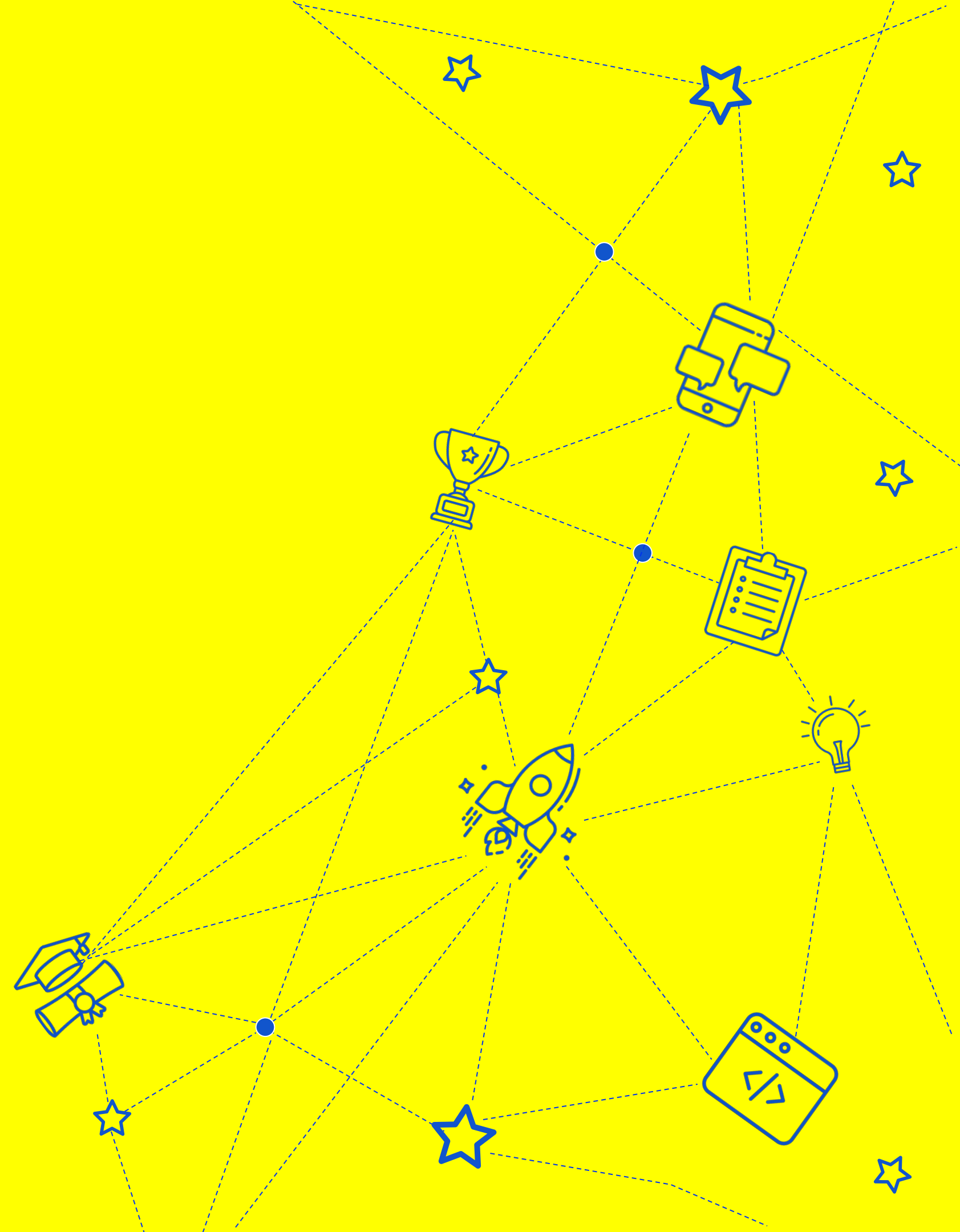
синхронизировать потоки



контролировать порядок
их выполнения



Синхронизация потоков. Класс Timer



Посмотрим на пример кода



Синхронизация потоков.

Класс Event



Что если два разных
потока пытаются **получить**
доступ к одним и тем же
данным?



Синхронизация потоков. Класс Event

Event или Событие



С помощью событий мы можем **оповещать**, выполнены какие-нибудь условия внутри потока или нет.



Посмотрим на пример кода



Синхронизация потоков. Класс Semaphore

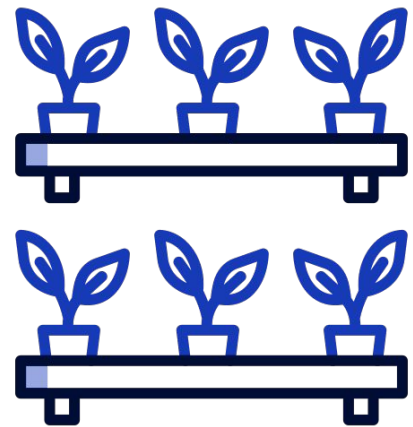


**Что если два разных
потока пытаются **работать**
в одно и то же время?**

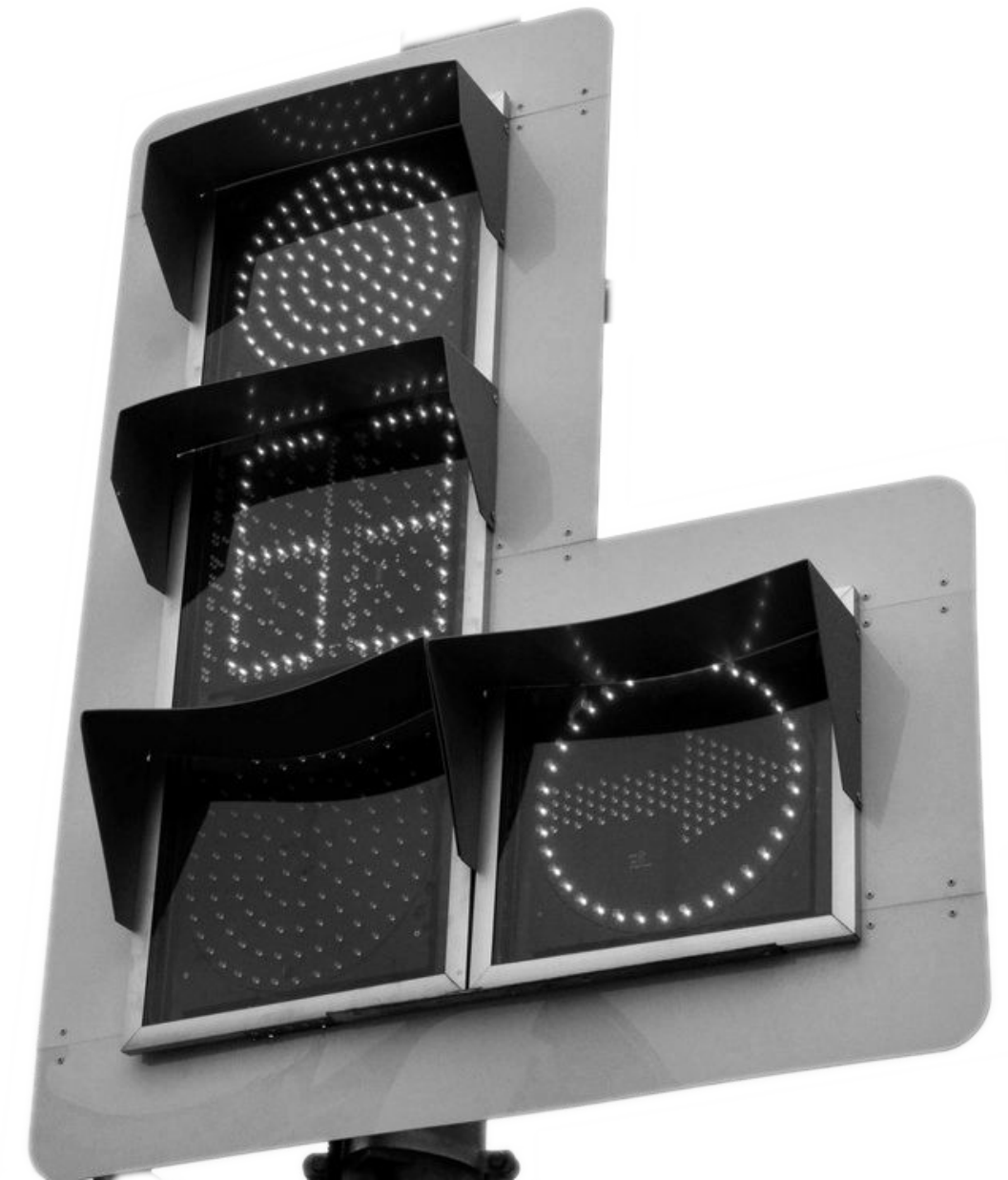


Синхронизация потоков. Класс Semaphore

Semaphore или Светофор



Позволяет задать определенное **количество мест** для потоков. Пока эти места заняты остальные **потоки будут ждать**, пока место освободится.



Посмотрим на пример кода



Синхронизация потоков.

Класс Barrier



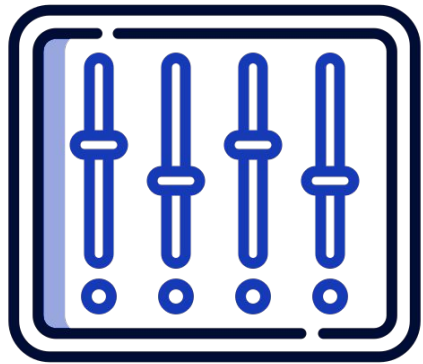
Что если нам нужно
дождаться определенного
состояния для всех
потоков?



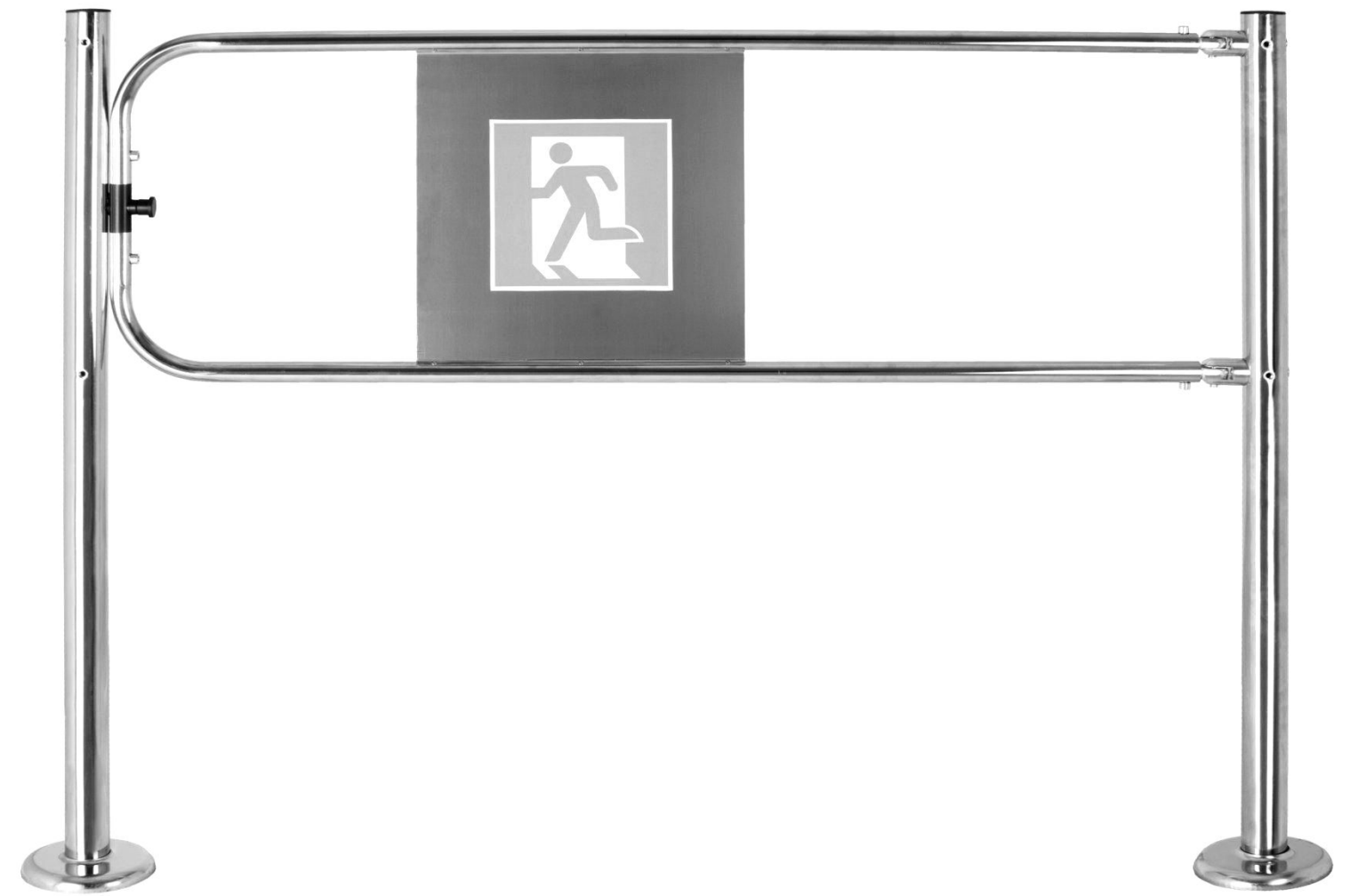
Синхронизация потоков.

Класс Barrier

Barrier или Барьер



Позволяет задать определенное **условие**, к которому должны прийти все потоки. После этого мы **можем выполнить** следующее действие с потоками.



Посмотрим на пример кода



Синхронизация потоков.

Класс Lock



Что если разные потоки
имеют доступ к одной
и той же переменной
и **могут изменять** ее?



Синхронизация потоков.

Класс Lock

Lock или Блокировщик

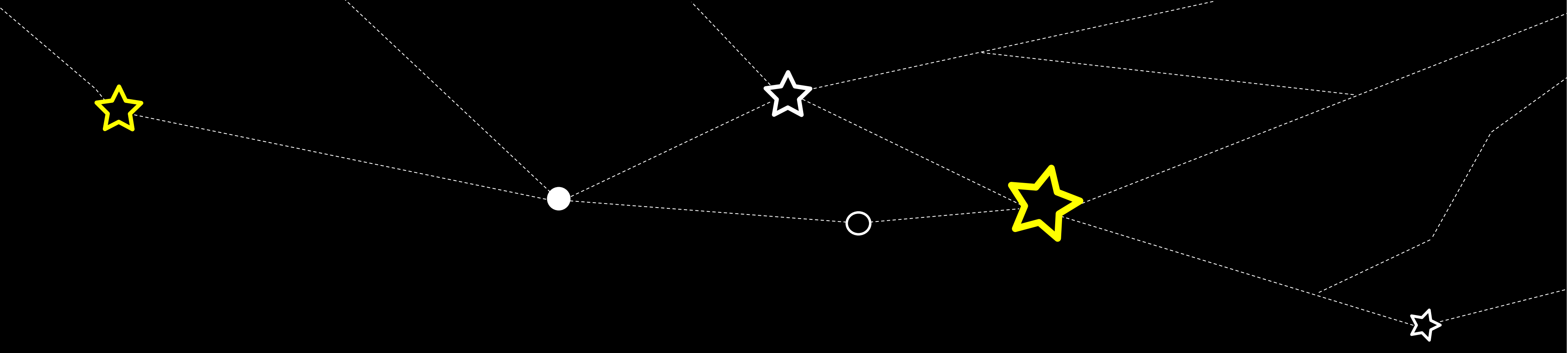


Если несколько потоков могут менять переменную, то нужно предусмотреть возможность **блокировки доступа** к этой переменной то для одного, то для второго потока по очереди.



Посмотрим на пример кода





СПАСИБО ЗА ВНИМАНИЕ

Василиса Серякова

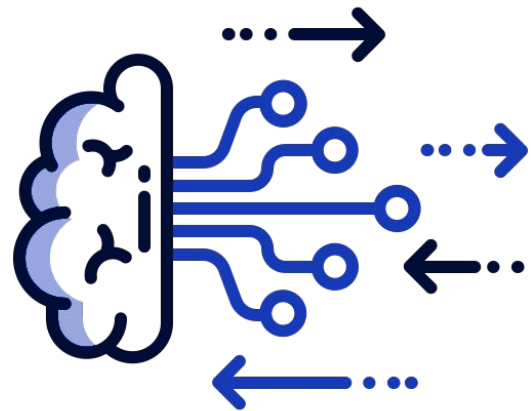


@vasilisa_seryakova



[Lady_in_ds](#)

Домашнее задание



Я вам предлагаю придумать новую легенду для **задачи**, которую мы **реализовали** на воркшопе.

Вместо работы Starbucks вы можете реализовать очередь в аптеке или супермаркете. Любую историю, которую вы придумаете.

Успехов!

